

Relatório de Benchmark e Otimização de API REST

25 de maio de 2026

1 Introdução

Este relatório apresenta os resultados obtidos durante os testes de desempenho realizados em uma API REST antes e após a aplicação de otimizações.

Os testes tiveram como objetivo analisar:

- Tempo médio de resposta;
- Throughput da aplicação;
- Estabilidade sob carga;
- Percentis de resposta;
- Quantidade de erros.

Os testes foram executados utilizando carga simultânea sobre múltiplos endpoints da aplicação:

- `/api/recurso-lento` — 40% dos acessos;
- `/api/recurso-detalle/{id}` — 30% dos acessos;
- `/api/status` — 20% dos acessos;
- `/api/recurso` — 10% dos acessos.

Entretanto, o foco principal da análise foi o endpoint `/api/recurso-lento`, pois ele representava o principal gargalo de desempenho da aplicação antes das otimizações.

2 Resultados Antes da Otimização

2.1 Teste 1

Tabela 1: Resultados baseline 1

Endpoint	Requisições	Erros	Média (ms)
<code>/api/recurso-lento</code>	3693	0	220
<code>/api/status</code>	1841	0	6.63
Aggregated	9354	0	90.90

2.2 Teste 2

Tabela 2: Resultados baseline 2

Endpoint	Requisições	Erros	Média (ms)
/api/recurso-lento	3793	0	219.98
/api/status	1830	0	6.26
Aggregated	9373	0	92.95

2.3 Teste 3

Tabela 3: Resultados baseline 3

Endpoint	Requisições	Erros	Média (ms)
/api/recurso-lento	3693	0	220
/api/status	1841	0	6.63
Aggregated	9354	0	90.90

2.4 Teste 4

Tabela 4: Resultados baseline 4

Endpoint	Requisições	Erros	Média (ms)
/api/recurso-lento	3693	0	220
/api/status	1841	0	4
Aggregated	9354	0	90.90

2.5 Teste 5

Tabela 5: Resultados baseline 5

Endpoint	Requisições	Erros	Média (ms)
/api/recurso-lento	3793	0	219.98
/api/status	1830	0	6.26
Aggregated	9373	0	92.95

3 Resultados Após a Otimização

3.1 Teste 1

Tabela 6: Resultados do teste otimizado 1

Endpoint	Requisições	Erros	Média (ms)
/api/recurso-lento	3905	0	5.33
/api/status	2012	0	4.59
Aggregated	9919	0	5.04

3.2 Teste 2

Tabela 7: Resultados do teste otimizado 2

Endpoint	Requisições	Erros	Média (ms)
/api/recurso-lento	4028	0	5.39
/api/status	1957	0	4.66
Aggregated	9896	0	5.07

3.3 Teste 4

Tabela 8: Resultados do teste otimizado 4

Endpoint	Requisições	Erros	Média (ms)
/api/recurso-lento	4125	0	5.25
/api/status	1981	0	4.57
Aggregated	9931	0	4.97

3.4 Teste 5

Tabela 9: Resultados do teste otimizado 5

Endpoint	Requisições	Erros	Média (ms)
/api/recurso-lento	3959	0	5.36
/api/status	2023	0	4.55
Aggregated	9910	0	4.98

4 Análise do Endpoint Crítico

O endpoint `/api/recurso-lento` foi definido como foco principal da otimização por representar o maior gargalo da aplicação.

Antes das otimizações, esse endpoint apresentava tempo médio de resposta em torno de 220 ms, impactando diretamente a média global da API e aumentando significativamente os percentis de latência.

Após as otimizações implementadas, o tempo médio caiu para aproximadamente 5 ms, demonstrando remoção efetiva do gargalo.

5 Comparação de Desempenho

5.1 Tempo Médio de Resposta



Figura 1: Comparação do tempo médio de resposta do endpoint /api/recurso-lento

5.2 Throughput

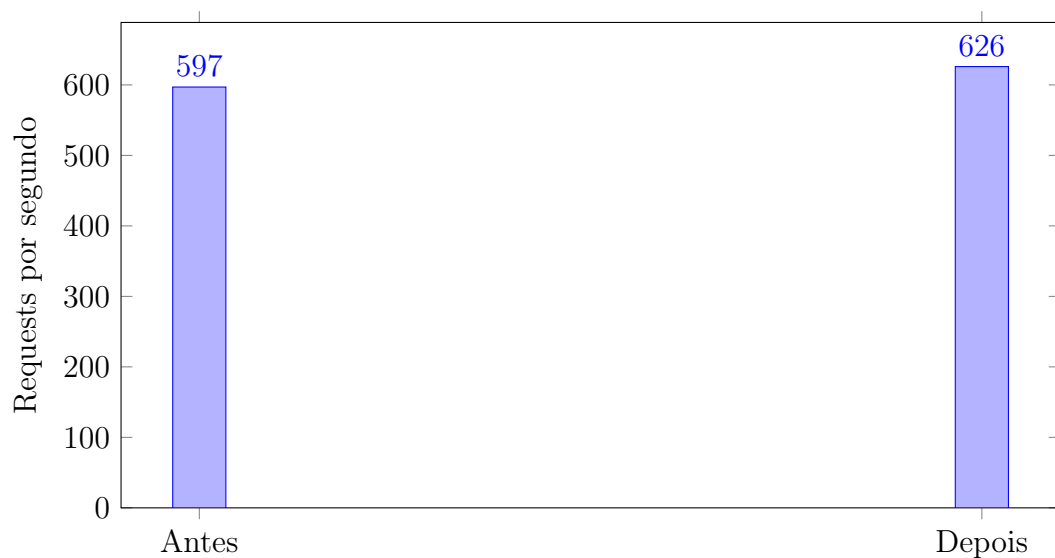


Figura 2: Comparação de throughput da aplicação

5.3 Redução Percentual de Latência

A redução percentual do tempo médio de resposta foi calculada utilizando:

$$\frac{220 - 5}{220} \times 100 \approx 97.7\%$$

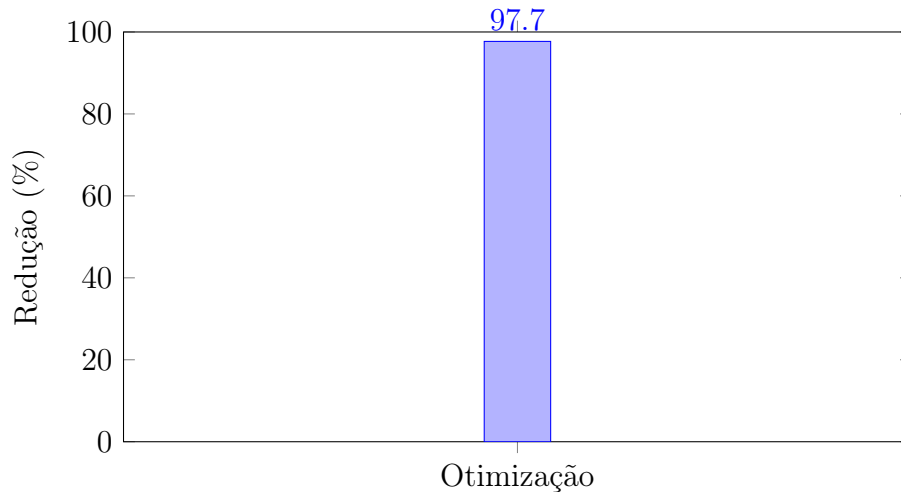


Figura 3: Redução percentual da latência após otimização

6 Análise dos Resultados

Após a aplicação das otimizações, observou-se uma melhoria significativa no desempenho da API.

Os resultados mostram que:

- O endpoint `/api/recurso-lento` reduziu sua latência média de aproximadamente 220 ms para cerca de 5 ms;
- O throughput permaneceu estável, apresentando inclusive pequeno aumento;
- Nenhum erro foi registrado durante os testes;
- Os percentis de resposta tornaram-se significativamente menores;
- A aplicação apresentou maior estabilidade sob carga.

A redução de aproximadamente 97.7% na latência indica que o principal gargalo da aplicação foi removido com sucesso.

7 Conclusão

Com base nos testes realizados, conclui-se que as otimizações aplicadas foram altamente eficazes.

A aplicação apresentou:

- Redução expressiva do tempo de resposta;

- Maior estabilidade;
- Melhor aproveitamento de recursos;
- Manutenção do throughput sob carga.

Os resultados demonstram que a API tornou-se significativamente mais eficiente após as melhorias implementadas.